Implementation of Rejection Sampling Methods for Generative Adversarial Networks

Emile Descroix, Anael Ernadotte

Année Universitaire 2025-2026

Contents

1 Introduction **3**

2 Metropolis-Hastings GANs **3**
2.1 Method Presentation 3
2.2 Key Points of the Algorithm 3

3 Optimal Budgeted Rejection Sampling **4**
3.1 Budget K and optimal acceptance function 4
3.2 Integration into GAN training 4
3.3 Implementation 5
3.4 Comparison with MH-GAN 5

4 Experimental Results and Comparison with other methods **5**
4.1 MH-GAN chain length impact 6
4.2 Soft and Hard Truncations 6
4.3 Methods Comparison 6

1 Introduction

This report aims to present, implements and compare two methods used to improve images generation using Generative Adversarial Networks (GANs). The two methods are based on rejection sampling, the original articles can be found at : Metropolis-Hastings Generative Adversarial Networks and Optimal Budgeted Rejection Sampling for Generative Models

For each method, we will start by presenting it, explain key points of its functioning and implementation and the potential adaptations needed for this project. Then, we will display and comment some experimental results.

Finally, we will compare our methods to a baseline version of GAN, and two other methods : Soft and Hard Truncations.

All the code used to implement methods and produce results is written in Python using PyTorch and the methods are being tested on the MNIST dataset.

2 Metropolis-Hastings GANs

2.1 Method Presentation

The Metropolis-Hastings GAN uses the trained discriminator to wrap the generator G into G' and improve samples quality without adding any training procedure. This method allows us to sample from the true distribution p_{data} using an imperfect generator with the condition that the discriminator D is perfect.

To draw samples we use Markov Chain Monte Carlo (MCMC) theory. We use a markov chains, $q(x'|x_k) = q(x')$, with K steps to approximate samples from our real data distribution. At step $k+1$, a sample x' is accepted with propability $\alpha(x', x_k) = min(1, \frac{p^*(x')q(x_k)}{p^*(x_k)q(x')})$ where p^* is our target distribution and q our sampler. If x' is accepted we have $x_{k+1} = x'$ else $x_{k+1} = x_k$.

In our case we use $p^* = p_D$, the distribution implied by D and $q = p_G$. Since p_D and p_G are unavaible for GANs we need to use the following relationship : $\frac{p_D}{p_G} = \frac{D(x)}{1-D(x)}$. This way, we can compute $\alpha(x', x_k)$ only using the evaluation of D. Our acceptance probability becomes

$$\alpha(x', x_k) = min\left(1, \frac{D(x_k)^{-1} - 1}{D(x')^{-1} - 1}\right)$$

Once x' is drawn, we also draw $u \sim \mathcal{U}(0,1)$, if $u < \frac{D(x_k)^{-1}-1}{D(x')^{-1}-1}$ then x' is accepted. We iterate this procedure K times.

Since MH-GAN is a rejection sampling method it has no impact on the training computational cost. However, generating samples is computationally heavier since we need to generate K images for each generated sample.

2.2 Key Points of the Algorithm

Initialization Since we have access to our data we can start each chain with a sample from the real distribution. This raises another issue : if after K steps no sample has been accepted x_K is a real image. To avoid this case MH-GAN allows to start a new chain with x_0 being a synthetic sample, this way we avoid outputting real data. Having a higher K reduces the probability of such a case but also means higher computational cost for each sample generated.

Discriminator Calibration When training a GAN the discriminator D tends to be really confident when classifying images and for every fake image the output would be really close to 0. This would result in a very low acceptance rate, since the initialization is a sample from the real distribution, and a small variety of images generated. Once the chain has moved to a fake sample, the values $D(x_k)$ and $D(x')$ would always be really close to 0. This would result in their ratio being noisy and uninformative, causing the chain to accept or reject randomly with no useful signal. To check the calibration of D one can use the statistic of

Dawid with $x_{i=1\dots n}$ samples and y_i the real/fake labels:

$$Z = \frac{\sum_{i=1}^{N} y_i - D(x_i)}{\sqrt{\sum_{i=1}^{N} D(x_i)(1 - D(x_i))}}$$

A well calibrated discriminator cannot distinguish y from $y \sim B(D(x))$ and we have $|Z| < 2$. If D is not well calibrated, one can use a held-out calibration using a set of evenly distributed fake/real examples.

This calibration also allows us to weaken the perfect discriminator hypothesis and accept an imperfect discriminator as long as the real/fake boundary is correct.

Accepting lower quality samples When considering a new sample, MH-GAN algorithm gives a small probability to get accepted even to images a lower discriminator score than the current sample x_k. While it may seems like hurting the quality of x_K it is essential in order to cover a wider range of the distribution. If we were to only accept samples with $\alpha(x', x_k) = 1$ the chain could get stuck in a local optimum. Accepting lower quality samples allow the chain more movement and enable the exploration of the real distribution.

3 Optimal Budgeted Rejection Sampling

3.1 Budget K and optimal acceptance function

In the framework of discriminator-based generative models, rejection sampling consists in accepting or rejecting samples generated by G according to an acceptance function $a : \mathcal{X} \to [0, 1]$. Without any budget constraint, one can set $a(x) = \frac{p(x)}{\hat{p}(x)M}$ with $M = \sup_x \frac{p(x)}{\hat{p}(x)}$, which guarantees $\tilde{P}_a = P$. However, in high dimension, M can be very large and make the acceptance rate negligible in practice.

The budget $K \geq 1$ represents the expected number of samples generated by G required to produce one accepted sample. The budget constraint writes $\mathbb{E}_{\hat{P}}[a(x)] \geq \frac{1}{K}$. For $K = 1$, no rejection is performed and $\tilde{P}_a = \hat{P}$. The larger K, the more selective and costly the rejection.

Optimal acceptance function. OBRS (Optimal Budgeted Rejection Sampling) seeks the function a minimizing the f-divergence $D_f(P\|\tilde{P}_a)$ under the budget constraint:

$$\min_a \; D_f(P\|\tilde{P}_a) \quad \text{s.t.} \quad \mathbb{E}_{\hat{P}}[a(x)] \geq \frac{1}{K}, \quad 0 \leq a(x) \leq 1 \tag{1}$$

Theorem 3.1 gives the explicit solution:

$$a_O(x) = \min\left(\frac{p(x)}{\hat{p}(x)} \cdot \frac{c_K}{M},\ 1\right) = \min\left(\gamma \cdot r(x),\ 1\right) \tag{2}$$

where $r(x) = p(x)/\hat{p}(x)$ is the density ratio, $\gamma = c_K/M$ is the Lagrange multiplier such that $\mathbb{E}_{\hat{P}}[a_O(x)] = \frac{1}{K}$. In practice, γ is computed by binary search over a mini-batch.

A remarkable result is that a_O does not depend on the choice of f: the optimal acceptance function is universal across all f-divergences.

3.2 Integration into GAN training

In classical GAN training, G is optimized to minimize $D_f(P\|\hat{P}_G)$, without accounting for the rejection that will occur after training. Yet the objective evaluated at the end is $D_f(P\|\tilde{P}_{a_O})$, which creates a mismatch. Training with OBRS (Tw/OBRS) addresses this by incorporating the rejection scheme into the training objective. To estimate $D_f(P\|\tilde{P})$ without requiring a second discriminator, the paper leverages the primal form of the f-divergence:

$$D_f(P\|\tilde{P}) = \mathbb{E}_{\hat{P}}\left[\frac{\tilde{p}(x)}{\hat{p}(x)} \cdot f\left(\frac{p(x)}{\hat{p}(x)}\right)\right] = \mathbb{E}_{\hat{P}}\left[K\, a_O(r(x)) \cdot f\left(\frac{r(x)}{K\, a_O(r(x))}\right)\right] \tag{3}$$

A single discriminator T is trained to estimate $r(x) = \nabla f^*(T^*(x)) = p(x)/\hat{p}(x)$ via the dual form of the f-divergence.

Theoretical benefits of Tw/OBRS. By incorporating rejection into training, $\hat{P}$ is pushed to cover the entire target distribution rather than a single mode, making the rejection procedure more effective. Furthermore, in practice, this flattens the loss landscape and reduces the number of suboptimal local minima.

3.3 Implementation

Three approaches were implemented and tested successively, in order of increasing complexity, but we only expose the results of the one of the paper : training and filtering with optimal budgeted rejection sampling.

Post-training rejection. The first approach consists in training a standard GAN, then applying OBRS only at generation time. Once the model is trained, K times more images than needed are sampled and accepted with probability $a_O(r(x))$. Samples with a high ratio $r(x)$ — where p_{data} is large relative to $\hat{p}_G$ — are accepted with probability close to 1. Those in regions poorly covered by G are rejected, improving the overall precision of the generated distribution without modifying training.

Training with $a_O(x)$ weighting. The second approach integrates OBRS into training by weighting the standard loss $-\log D(G(z))$ by the weights $a_O(r(x))$. This approach focuses learning on samples deemed promising by D, unlike classical BCE which weights samples solely by $-\log D(x)$ — a quantity measuring how much D is fooled, not how useful the sample is for covering p_{data}.

Training with the paper's loss function. The third approach implements the full TwOBRS loss depending on f:

$$\mathcal{L}_G = \mathbb{E}\left[K \cdot a_O(r(x)) \cdot f\left(\frac{r(x)}{K \cdot a_O(r(x))}\right)\right] \tag{4}$$

This loss differs from simple weighting when some samples reach the clamping threshold $a_O(r(x)) = 1$. For these samples, the term $f(r(x)/K)$ grows nonlinearly with $r(x)$, amplifying their contribution well beyond what linear weighting allows. This amplification encourages G to refine regions of the space it already covers well, rather than simply correcting its worst samples as BCE would.

However, this approach required constraining the discriminator to prevent it from dominating G. Two adjustments were necessary to stabilize training: a differential learning rate ($\text{lr}_D = 10^{-4}$, $\text{lr}_G = 2 \times 10^{-4}$) and a reduced update frequency for D, trained every other iteration.

3.4 Comparison with MH-GAN

MH-GAN and OBRS both exploit the trained discriminator to improve generation quality without modifying the generator, but they differ fundamentally in their approach. MH-GAN uses a Markov chain of K steps to iteratively refine each sample, it is grounded in MCMC theory and provably converges to p_{data} with a perfect discriminator. However, it requires calibration to function correctly and becomes computationally expensive at high K. OBRS takes a different approach: rather than iterating, it computes a single optimal acceptance decision per sample, clipping the density ratio at a threshold γ determined by the budget K. This makes OBRS faster and theoretically stronger since it is provably optimal for any f-divergence under a fixed budget, and Theorem 3.3 guarantees that Precision improves by a factor K without any loss in Recall. A further distinction is that OBRS can be integrated into training (Tw/OBRS), pushing the generator itself toward mass-covering distributions, whereas MH-GAN operates purely at inference time and is therefore bounded by the quality of the pre-trained generator.

4 Experimental Results and Comparison with other methods

In this section, we present experimental results for both methods and compare them with soft and hard truncations. To evaluate our models we use three metrics : Fréchet Inception Distance (FID), Precision and Recall. We also compute the time needed to generate the samples since it represents one of the limit typically for MH-GAN.

We test our models using the MNIST dataset and a GAN with four fully connected layers and leaky ReLu activation for both the generator and the discriminator. Results are obtained by averaging output of five runs each generating ten thousands samples.

4.1 MH-GAN chain length impact

We start by illustrating the impact of the chain length when using the MH-GAN method. On the graph below, see Fig.1, we notice that FID decreases rapidly as K is growing. When allowing more steps per sample generated the chain has more opportunities to move toward samples the discriminator considers more realistic. However, the FID hits a plateau past $K = 50$, suggesting it could be the sweet spot for our architecture and dataset.

We also notice a usual precision/recall tradeoff, when increasing K we tend to accept lower quality samples since with longer chains the MCMC sampler explores the full distribution more broadly, which improves diversity but reduces the concentration of samples in high-quality regions.

Finally, the time needed to produce a fixed amount of samples grows linearly with the chain length since we need to generate K images for each sample. This shows that if generation time matters in a specific problem we might want to settle with shorter chains since higher K offers little to no improvement in FID and precision and marginal gains in recall.

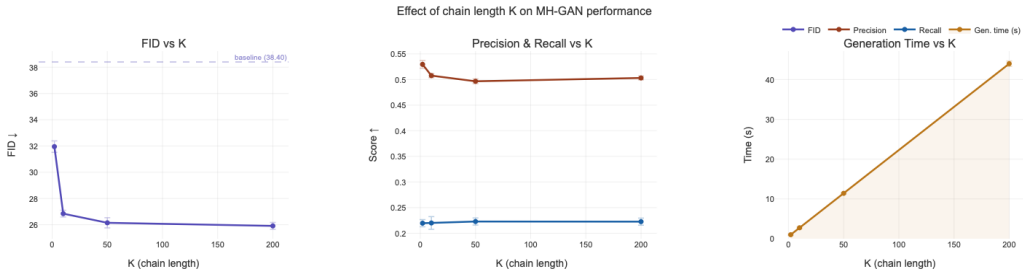


Figure 1: Impact of chain length on different metrics.

4.2 Soft and Hard Truncations

Soft and Hard truncations methods aims to reduce the spread of the distribution since the tails are harder to generate for the models. To do so they use similar methods to concentrate the distribution around its peak(s).

Hard truncation sets a threshold based on parameter ψ and rejects any z outside of it. This way the model does not generate any sample corresponding to the tails of the distribution resulting in an improved precision with the cost of hurting the recall.

Soft truncation uses a different approach by multiplying z by $\psi \in [0, 1]$, this way every sample is brought closer to the center of the distribution. This method have a similar effect as hard truncation.

4.3 Methods Comparison

The baseline achieves a FID of 32.40, a precision of 0.511 and a recall of 0.210, serving as the reference point for all comparisons.

MH-GAN consistently improves FID over the baseline across all values of K, dropping from 31.95 ($K = 2$) to 25.91 ($K = 200$). Recall also increases with K ($0.221 \rightarrow 0.292$), while precision slightly decreases ($0.553 \rightarrow 0.521$), reflecting a classic precision/recall trade-off. However, this quality gain comes at a significant computational cost: $K = 200$ requires 44 s per generation, over $400\times$ slower than the baseline.

Hard Truncation ($\psi = 0.7$) achieves the best recall in the table (0.312), but at the expense of a degraded FID (36.84, worse than baseline) and the lowest precision (0.405). It favours sample diversity over overall quality.

Table 1: Comparison of generation methods, results averaged over 5 runs. Best row for each metric in **bold**.

Method	Params	FID ↓	Precision ↑	Recall ↑	Gen. Time (s)
Baseline	—	32.40 ± 0.53	0.511 ± 0.007	0.210 ± 0.005	0.1 ± 0.0
MH-GAN	$K = 2$	31.95 ± 0.44	0.553 ± 0.008	0.221 ± 0.007	1.0 ± 0.2
	$K = 10$	26.85 ± 0.26	0.525 ± 0.004	0.241 ± 0.012	2.8 ± 0.2
	$K = 200$	25.91 ± 0.26	0.521 ± 0.004	0.292 ± 0.007	44.0 ± 0.7
Hard Truncation	$\psi = 0.7$	36.84 ± 0.61	0.405 ± 0.004	**0.312 ± 0.005**	0.1 ± 0.0
Soft Truncation	$\psi = 0.5$	63.37 ± 0.36	0.551 ± 0.004	0.119 ± 0.005	0.1 ± 0.0
	$\psi = 0.7$	42.41 ± 0.41	0.471 ± 0.004	0.208 ± 0.015	0.1 ± 0.0
	$\psi = 0.9$	36.38 ± 0.42	0.421 ± 0.004	0.286 ± 0.006	0.1 ± 0.0
Tw/OBRS	$K = 2$	42.6 ± 0.6	**0.56 ± 0.01**	0.31 ± 0.02	0.5 ± 0.1
	$K = 5$	29.6 ± 0.2	0.51 ± 0.01	0.3 ± 0.02	2.5 ± 0.3
	$K = 10$	**23.7 ± 0.2**	0.48 ± 0.01	0.3 ± 0.01	5.1 ± 0.7

Soft Truncation is highly sensitive to ψ. At $\psi = 0.5$, FID degrades severely (63.37) and recall collapses to 0.119. Increasing ψ to 0.9 partially recovers both metrics, but the method never outperforms the baseline on FID, making it the weakest approach overall.

Tw/OBRS yields the strongest results. At $K = 10$, it achieves the best FID in the table (23.7), outperforming MH-GAN ($K = 200$) while being roughly 9× faster (5.1 s vs. 44.0 s). At $K = 2$, it reaches the best precision (0.560), making it the method of choice when sample fidelity is the primary concern. Recall stabilises around 0.30 for $K \geq 5$, regardless of K.

Overall, Tw/OBRS offers the best quality/cost trade-off, and should be preferred in most settings. MH-GAN remains a competitive alternative when compute is not a constraint. Truncation-based methods are generally not recommended, except when maximising recall specifically (Hard Truncation).

References

[1] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative adversarial nets. In *Advances in Neural Information Processing Systems (NeurIPS)*, 27, 2672–2680.

[2] Turner, R., Hung, J., Frank, E., Saatci, Y., & Yosinski, J. (2019). Metropolis-Hastings generative adversarial networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, *Proceedings of Machine Learning Research*, 97, 6345–6353.

[3] Verine, A., Pydi, M. S., Negrevergne, B., & Chevaleyre, Y. (2024). Optimal budgeted rejection sampling for generative models. In *Proceedings of the 27th International Conference on Artificial Intelligence and Statistics (AISTATS)*, *Proceedings of Machine Learning Research*, 238.

[4] Kynkäänniemi, T., Karras, T., Laine, S., Lehtinen, J., & Aila, T. (2019). Improved precision and recall metric for assessing generative models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 32.

[5] Karras, T., Laine, S., & Aila, T. (2019). A style-based generator architecture for generative adversarial networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 4401–4410.

[6] Kingma, D. P., & Dhariwal, P. (2018). Glow: Generative flow with invertible 1×1 convolutions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 31, 10215–10224.

[7] Azadi, S., Olsson, C., Darrell, T., Goodfellow, I., & Odena, A. (2019). Discriminator rejection sampling. In *Proceedings of the 7th International Conference on Learning Representations (ICLR)*.

[8] Sajjadi, M. S. M., Bachem, O., Lucic, M., Bousquet, O., & Gelly, S. (2018). Assessing generative models via precision and recall. In *Advances in Neural Information Processing Systems (NeurIPS)*, 31, 5228–5237.

[9] Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2818–2826.

[10] Nowozin, S., Cseke, B., & Tomioka, R. (2016). f-GAN: Training generative neural samplers using variational divergence minimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 29, 271–279.